exec.pyc

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 38s >

Este arquivo é malware (loader/downloader com C2 via blockchain).

Análise passo a passo

1. **Tipo e cabeçalho do arquivo**
   É um bytecode Python compilado (`.pyc`) para **Python 3.12** (magic number `f30d0d0a`).
   O nome original do fonte embutido é um hash longo (`1e6487b7fbb9a0ff4c1d7032bfecd153fa6b9cc3c523243e8183588a98168267.py`), típico de código ofuscado ou gerado automaticamente.
2. **Imports e nomes de funções/variáveis**
   O módulo importa de forma legítima bibliotecas comuns (`os` , `sys` , `json` , `subprocess` , `urllib` , `socket` , `zipfile` , `shutil` , `pathlib` , `asyncio` , `platform` , `base64` , etc.).
   Porém os nomes das funções principais são deliberadamente ofuscados:
   - `fvzsrnzlh`
   - `scyzzvvy`
   - `_getSignFAddress`
   - `_isRussianSystem`
   - `download_nodejs`
   - `main` + callback interno
3. **Verificação de sistema russo (`_isRussianSystem`)**
   O código verifica de várias formas se o sistema parece russo:
   - variáveis de ambiente (`USERNAME` , `LANG` , `LANGUAGE` , `LC_ALL`)
   - locale (`ru_RU` , `ru-RU` , “russian”, “россия”, “русский”)
   - timezone (`/etc/timezone` , `/etc/localtime` , zonas de Moscou, Kaliningrado, Samara, etc.)
   - no Windows usa `tzutil`
     Se detectar sistema russo, **pula a execução** e imprime “Russian system detected, skipping execution”.
     Essa técnica de geofencing (evitar vítimas em países da CEI) é clássica em malware de grupos de cibercrime que atuam fora da Rússia/Bielorrússia.
4. **C2 via blockchain Solana**
   - Função `_getSignFAddress` : consulta endpoints públicos da Solana (`api.mainnet-beta.solana.com` e outros) via JSON-RPC `getSignaturesForAddress` .
   - Endereço fixo usado: `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` .

- Função `fvzsrnzlh` : extrai o campo **memo** das transações, remove prefixos numéricos e faz parse de JSON.
  Isso é um canal de comando e controle (C2) baseado em blockchain — técnica usada para dificultar bloqueio e rastreamento.

5. **Download e execução de Node.js + payload JS**
   - `download_nodejs` : baixa Node.js oficial (v22.9.0) para Windows/macOS/Linux (x64/arm64), extrai e localiza o executável.
   - `scyzzvvy` : baixa dados de uma URL (com headers específicos), extrai `iv` e `secretKey` em base64.
   - No `callback` (dentro de `main`):
     - Recebe o payload (link + dados criptografados/base64).
     - Monta um arquivo `i.js` que contém:

       ```javascript
       var https = require("https");
       const secretKey = '...';
       const _iv = Buffer.from('...', "base64");
       eval(atob('...'));
       ```

     - Executa o Node.js em background (`subprocess.Popen` com `DEVNULL` e, no Windows, `CREATE_NO_WINDOW`).
       O uso de `eval(atob(...))` + Node.js é padrão de loaders que entregam stealers, RATs ou outros payloads ofuscados.
6. **Controle de execução / persistência leve**
   - Usa `~/init.json` com data de expiração (geralmente 2 dias).
   - Se o arquivo ainda for válido, pula a execução.
   - Caso contrário, continua o fluxo de busca de memo na Solana → download → execução.

Conclusão

O arquivo **não é benigno**. Trata-se de um **loader/downloader** ofuscado que:

- Evita sistemas russos,
- Usa a blockchain Solana como canal C2 (via memos),
- Baixa Node.js se necessário,
- Descriptografa/decodifica e executa um payload JavaScript ofuscado.

Comportamento típico de malware distribuído em campanhas recentes que abusam de blockchains públicas para C2.

Recomendação: não execute o arquivo em nenhum ambiente real. Trate-o como amostra de malware.

Qual é a categoria principal deste malware?

Worked for 6s >

Categoria principal: Loader (Downloader / Stager)

Este malware atua fundamentalmente como um **loader**.

Por que Loader?

- Sua função principal é **baixar e executar um payload secundário** (o código JavaScript ofuscado que é montado no arquivo `i.js` e executado via Node.js com `eval(atob(...))`).
- Ele não realiza ações maliciosas finais por conta própria (roubo de dados, ransomware, mineração, etc.). Em vez disso, serve como estágio intermediário.
- Usa a blockchain Solana (memos de transações) como canal de C2 para obter o próximo estágio.
- Baixa o interpretador necessário (Node.js) caso não esteja presente.
- Possui lógica de controle de reexecução (`init.json`) e geofencing (evita sistemas russos).

Classificação mais precisa

Categoria	Aplica?	Observação
Loader / Downloader	Sim	Categoria principal
Dropper	Parcial	Escreve o payload em disco antes de executar
Trojan	Genérico	Classificação ampla, menos específica
Stealer / RAT / etc.	Não	Isso seria o possível payload final (desconhecido)

Resumo: Trata-se de um **loader com C2 baseado em blockchain (Solana)**, projetado para entregar e executar um segundo estágio em JavaScript.

↘ Qual é o nome do malware

↘ Como funciona o C2 via blockchain?

↘ Quais são as categorias secundárias?